Getting Started

This guide walks through building the workspace, starting local brokers, launching a minimal pipeline, and verifying that messages are flowing.

Prerequisites

- ROS 2 (Humble or later) installed and sourced.
- `colcon`, `rosdep`, and common ROS 2 build tools available.
- Docker + Docker Compose for running local Kafka / MQTT brokers.
- `kcat` (formerly `kafkacat`) for quick Kafka verification — install with `sudo apt install kafkacat` or `brew install kcat`.

---

1. Build the workspace

Clone the repository into your ROS 2 workspace `src/` directory, then:

```
# From the workspace root (parent of src/)
rosdep install --from-paths src --ignore-src -y
colcon build --symlink-install \
             --cmake-args -DCMAKE_BUILD_TYPE=Release \
                          -DCMAKE_EXPORT_COMPILE_COMMANDS=On
source install/setup.bash
```

Rebuilding only the dispatcher packages during iteration:

```
colcon build --packages-up-to dispatcher_controller kafka_sink int
source install/setup.bash
```

---

2. Start local brokers

Kafka

```
 cd kafka_bridge/kafka_brocker
docker compose up -d
```

Verify the broker is up:

```
 kcat -b localhost:9092 -L
```

MQTT (optional)

```
 cd mosquitto_bridge/mosquitto_brocker
docker compose up -d
```

---

3. Create a topic selection file

Create a YAML file listing the ROS 2 topics you want to forward. Example:

```
 # ~/topics.yaml
- topic_name: /chatter
  msg_type: std_msgs/msg/String

- topic_name: /odom
  msg_type: nav_msgs/msg/Odometry
```

You can omit `msg_type` when `validate_topics:=false` (the default) and the topics are already publishing — the introspection manager will infer the type.

An example file is provided at:

```
ros2_kafka_dispatcher_bringup/config/selection_example.yaml
```

---

4. Launch the minimal pipeline

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  selection_mode:=file \
  selection_file_path:=/absolute/path/to/topics.yaml
```

This starts four nodes:

Node	Role
`introspection_manager`	Monitors the ROS 2 graph, infers message types
`dispatcher_controller`	Loads the selection file, drives sink lifecycles
`kafka_sink`	Forwards selected topics to Kafka
`mosquitto_sink`	Forwards selected topics to MQTT

To disable the MQTT sink if you only need Kafka:

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  selection_mode:=file \
  selection_file_path:=/absolute/path/to/topics.yaml \
  enable_mosquitto_sink:=false
```

---

5. Publish a test message

In a separate terminal (source your workspace first):

```
ros2 topic pub /chatter std_msgs/msg/String "data: 'hello kafka'"
```

---

6. Verify the pipeline

Check controller status

```
ros2 service call /dispatcher_controller/get_status \
  dispatcher_controller/srv/GetStatus "{}"
```

Expected response when streaming is active:

```
phase: IDLE
selection_mode: file
lifecycle_state_kafka: active
applied_topics_count: 2
last_error: ''
```

Check topic discovery

```
ros2 topic echo /introspection_manager/topics_info
```

Confirm messages arrived in Kafka

```
kcat -b localhost:9092 -t ros2.chatter -C -e
```

The Kafka topic name follows the pattern `<topic_prefix>.<ros_topic_without_leading_slash_dots>`. With the default prefix `ros2`, `/chatter` becomes `ros2.chatter`.

---

7. Common adjustments

Switch to all-topics mode

Automatically forward every topic in the graph:

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  selection_mode:=all \
  enable_mosquitto_sink:=false
```

Set `all_mode_max_topics` to a safe limit in your parameter file when using this mode in a dense graph.

Enable topic validation

Verify that each topic in the selection file is actively publishing before activating sinks:

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  selection_mode:=file \
  selection_file_path:=/path/to/topics.yaml \
  validate_topics:=true
```

Reload the selection at runtime

After editing `topics.yaml`, apply the changes without restarting:

```
ros2 service call /dispatcher_controller/reload_selection \
  dispatcher_controller/srv/ReloadSelection "{}"
```

Stop streaming

```
ros2 service call /dispatcher_controller/stop_streaming \
  dispatcher_controller/srv/StopStreaming "{}"
```

---

8. Troubleshooting

Symptom	What to check
`kafka_sink` stays `inactive`	

Symptom	What to check
	Broker unreachable — run `kcat -b localhost:9092 -L` and check `kafka.bootstrap_servers`.
`get_status` shows `ERROR`	Inspect `last_error` in the response.
Topics missing from `topics_info`	Ensure the publisher is running and `filter_hidden:=true` isn't hiding it (topics with `_`-prefixed segments are hidden by default).
Kafka topic not created	The topic is created on first produce — check that a message was actually published on the ROS 2 side.
`msg_type` required error	Set `validate_topics:=true` only when the introspection manager can see the topic, or provide `msg_type` explicitly in the selection file.

---

9. Where to go next

Document	Description
Architecture	How the components interact and where data flows.
Configuration Reference	Every parameter for every node.
API Reference	Services, published topics, Kafka record format.
Deployment	Docker build, production parameter files, CI/CD.

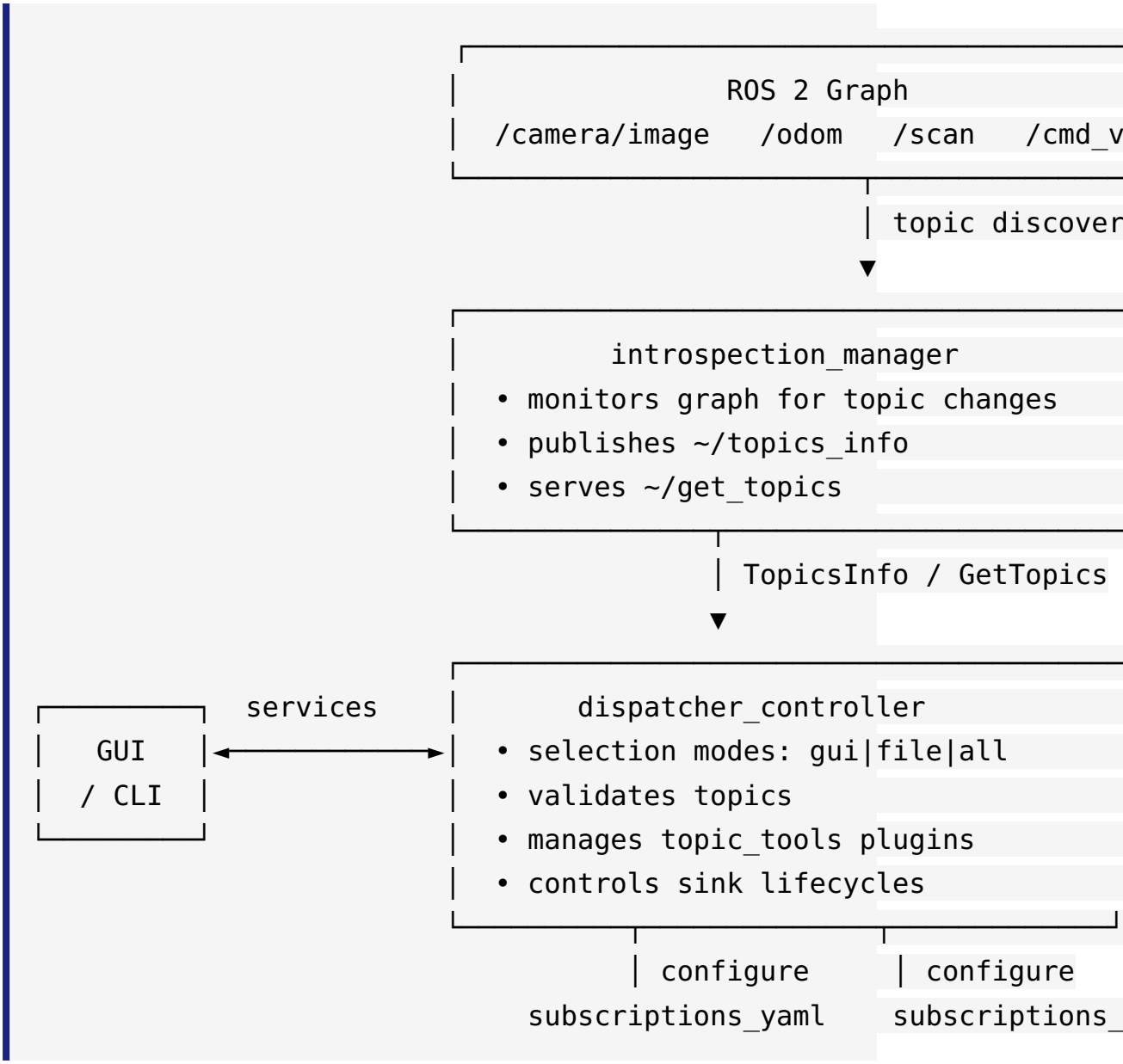
Document	Description
Developer Guide	Adding sinks, plugins, contributing.
Latency Measurement	Measuring end-to-end latency with the built-in tools.
Benchmark Plan	Structured test matrix and procedure.

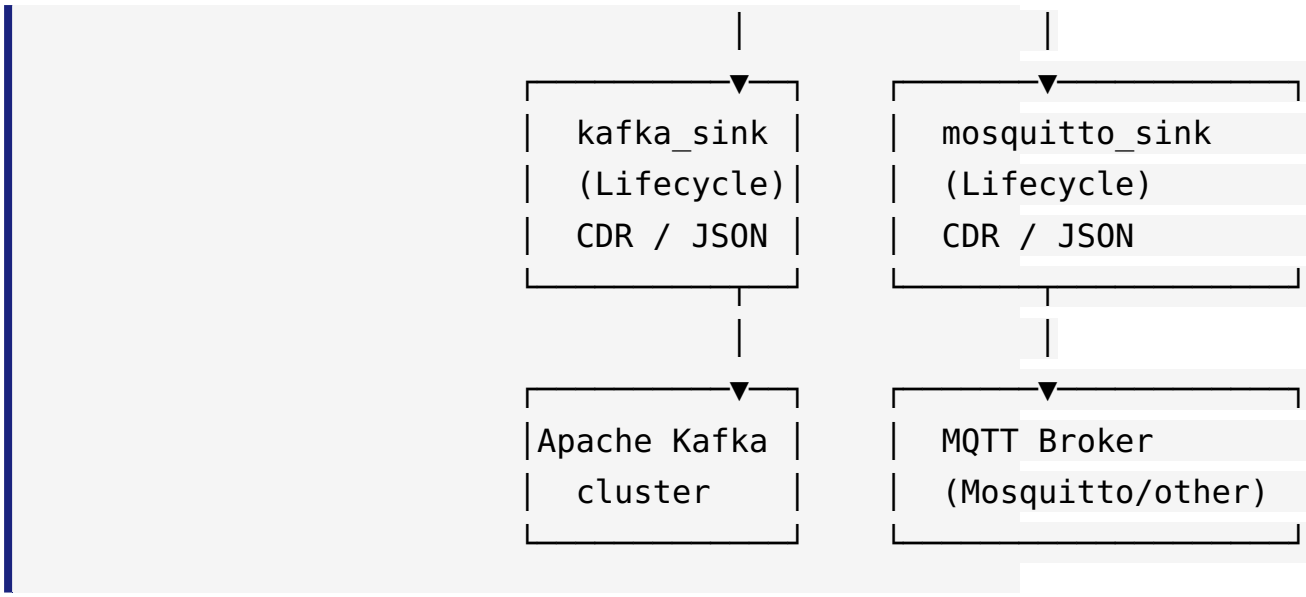
Architecture

This document describes the system design, component interactions, and data flow of `ros2_kafka_dispatcher`.

Overview

`ros2_kafka_dispatcher` bridges ROS 2 topics to external message brokers (Apache Kafka and MQTT). It provides dynamic topic discovery, flexible selection modes, and lifecycle-managed sinks.





Components

introspection_manager

A standard `rclcpp::Node` that continuously monitors the ROS 2 graph.

- Runs a background thread that polls for topic/type changes.
- Publishes discovered topics on `~/topics_info` (latched, transient-local by default).
- Serves `~/get_topics` for on-demand queries.
- Filters hidden topics (name segments starting with `_`) by default.

When to use it: It is required whenever `dispatcher_controller` needs to infer message types (e.g. `validate_topics:=true` or `selection_mode:=all`).

dispatcher_controller

The control-plane node. It is a standard `rclcpp::Node` that orchestrates everything else.

Selection modes:

Mode	Description
`file`	Reads topic list from a YAML file on disk (default).

Mode	Description
`gui`	Waits for an `apply_selection` service call from an external client (e.g. rqt GUI).
`all`	Auto-discovers all topics via `introspection_manager` (subject to allowlist/denylist/max).

Lifecycle orchestration:

1. Receives a topic selection (from file, GUI call, or auto-discovery).
2. Optionally validates each topic against the live graph via `introspection_manager`.
3. Serialises the selection to `subscriptions_yaml` and calls the lifecycle transitions on the Kafka and Mosquitto sinks: `configure` → `activate`.
4. On `stop_streaming`, calls `deactivate` on all sinks.

topic_tools integration:

For each topic in the selection, an optional `topic_tools` block can specify a plugin. The controller loads the plugin as a component into the shared container (`component_container_name`) via `composition_interfaces/srv/LoadNode`, wires the ROS topic through it (`input_topic` → plugin → `output_topic`), and registers the transformed output topic with the sink instead of the original.

The integration model is strictly **one input topic → one output topic**. Plugin names follow the `package::ClassName` convention and are passed directly to the component loader without validation, so any composable node that conforms to the single-input/single-output contract can be used.

Available topic_tools nodes:

Node	Plugin name	Key parameters	Support status
Throttle	`topic_tools::ThrottleNode`		

Node	Plugin name	Key parameters	Support status
		`period` (s between messages)	Supported — tes config examples available
Drop	`topic_tools::DropNode`	`x` (drop 1-in-x messages)	Supported — liste developer guide, config example y
Delay	`topic_tools::DelayNode`	`delay` (seconds)	Supported — liste developer guide, config example y
Relay	`topic_tools::RelayNode`	—	Architecturally compatible (singl → single out); no tested or docume
RelayField	`topic_tools::RelayFieldNode`	`expression` (field path), `output_type`	Architecturally compatible; requi `output_type` config; not tested documented
Transform	`topic_tools::TransformNode`	`expression`, `output_type`	**Not supported** – Python-only node ROS 2 Humble+; cannot be loaded C++ component `LoadNode`
Mux	`topic_tools::MuxNode`	`mux_topics` (list)	**Not supported** – requires multiple topics; current `TopicToolsCo` maps exactly one `input_topic`
Demux	`topic_tools::DemuxNode`		

Node	Plugin name	Key parameters	Support status
		`demux_topics` (list)	**Not supported** – requires multiple output topics; cur `TopicToolsCo` maps exactly one `output_topic`

Node descriptions:

- **Throttle** — Forwards messages from `input_topic` to `output_topic` at most once every `period` seconds. Excess messages are silently dropped. Useful for reducing Kafka ingestion rate for high-frequency sensors (e.g. lidar at 20 Hz → 5 Hz).
- **Drop** — Forwards every x-th message and discards the rest. Unlike Throttle, Drop is count-based rather than time-based, so the output rate tracks the input rate proportionally. Useful when a fixed decimation ratio is needed regardless of timing jitter.
- **Delay** — Buffers incoming messages and republishes them after a fixed `delay` in seconds. Does not change message rate or content. Useful for time-aligning topics from sensors with different hardware latencies before sending to Kafka.
- **Relay** — Re-publishes messages from `input_topic` to `output_topic` without any modification. Primarily useful for renaming a topic or bridging across namespace boundaries before forwarding to a sink.
- **RelayField** — Extracts a single field from the incoming message and publishes it as a new message on `output_topic`. The field is selected via a Python-style `expression` (e.g. `m.linear.x`). The `output_type` must be set to the resulting message type. Useful for stripping large messages down to a single scalar or sub-message before sending to Kafka.

- **Transform** *(not supported)* — Applies an arbitrary Python expression to the incoming message and publishes the result. Implemented as a pure Python node in ROS 2 Humble+; it is not registered as a composable C++ component and therefore cannot be loaded via `composition_interfaces/srv/LoadNode`. Cannot be used with `dispatcher_controller`.

- **Mux** — Subscribes to a list of input topics and republishes the selected one to a single output topic. The active input can be switched at runtime via a service call. Not supported because the current `TopicToolsConfig` struct models exactly one `input_topic`; extending support would require a `mux_topics` list field and corresponding YAML parsing.

- **Demux** — Routes messages from a single input topic to one of several output topics, switchable at runtime. Not supported because the current model maps to exactly one `output_topic`; support would require a `demux_topics` list and the sink would need to know which output channel to subscribe to.

kafka_sink

A `rclcpp_lifecycle::LifecycleNode` that forwards ROS 2 messages to Apache Kafka.

- On `configure`: parses `subscriptions_yaml` and creates a `KafkaProducer`.
- On `activate`: creates generic ROS 2 subscriptions (using `rosidl` type introspection) and starts the producer poll thread.
- On `deactivate`: destroys subscriptions; producer drains and stops.
- Each message is serialized (CDR or JSON) and sent to a Kafka topic derived from the ROS topic name (prefix or fixed mapping).
- Adds one Kafka record header: `ros_type` (the ROS 2 message type). Other fields (`ros_topic`, `kafka_topic`, `stamp_ms`, `payload_format`) are written to the telemetry log, not set as headers.

- Optionally publishes per-topic metrics to ROS 2 at a configurable interval.

mosquitto_sink

Mirrors `kafka_sink` functionality for MQTT using the Eclipse Paho C++ client.

- Supports TLS/SSL, LWT (Last Will and Testament), QoS levels 0/1/2, and automatic reconnection.
- Topic naming follows the same prefix/fixed mapping as `kafka_sink`.

kafka_client (library)

A C++ library wrapping `librdkafka` with:

- Async producer with a bounded, drop-when-full queue.
- Background poll thread for delivery callbacks.
- `STRICT` startup mode (fail if broker unreachable) or `TOLERANT` (continue without broker).
- Header support.

kafka_source

A lifecycle node that consumes Kafka topics and republishes messages as ROS 2 topics.

- Subscribes via regex pattern.
- Deserializes CDR payloads using `rosbag2_cpp` type support.
- Used for round-trip latency testing and bridging Kafka back into ROS 2.

kafka_cdr_to_json

A lifecycle node that reads CDR-encoded Kafka records and republishes them as JSON to new Kafka topics (adds a `.json` suffix by default).

ros2_kafka_dispatcher_bringup

Contains system-level launch files and example configuration files. Two main launch files:

- `system_minimal.launch.py` — each node in its own process.
- `system_composed.launch.py` — all nodes as components in a single container.

Data flow: ROS 2 → Kafka

```
ROS 2 publisher
      │  raw message (DDS)
      ▼
kafka_sink subscription callback
      │  rosidl introspection serializes to CDR (or JSON)
      ▼
KafkaProducer (kafka_client)
      │  enqueues message + headers to bounded queue
      ▼
librdkafka poll thread
      │  batches and sends to broker
      ▼
Apache Kafka partition
      │  stored, replicated
      ▼
downstream consumers (kafka_source, analytics, ML pipelines, ...)
```

Data flow: Kafka → ROS 2 (kafka_source)

```
Apache Kafka partition
      │
      ▼
kafka_source consumer thread
      │  reads CDR bytes and the `ros_type` header
```

```
      ▼
rosbag2_cpp deserializer
      │  reconstructs typed ROS 2 message
      ▼
ROS 2 publisher → downstream subscribers
```

Lifecycle state machine

Both `kafka_sink` and `mosquitto_sink` use the standard ROS 2 lifecycle:

```
UNCONFIGURED ──configure──► INACTIVE ──activate──► ACTIVE
                              ▲                      │
                              └─────deactivate───────┘
                              │
                           cleanup
                              │
                              ▼
                          UNCONFIGURED
```

The `dispatcher_controller` drives these transitions. It maintains its own internal phase:

Phase	Meaning
`IDLE`	Ready to accept a new selection.
`BUSY`	Currently applying or stopping a selection.
`ERROR`	Last operation failed; inspect `last_error`.

Deployment topologies

Single-machine (development)

All nodes on one machine, Kafka running in Docker. Latency dominated by local loopback.

Edge device → cloud Kafka

Robot / edge device runs the full ROS 2 stack and the dispatcher. Kafka cluster lives in the cloud. Network latency adds to end-to-end measurement but CDR payload is compact.

Composed vs. separate processes

`system_composed.launch.py` runs everything in one component container — lower IPC overhead, single process to monitor. `system_minimal.launch.py` gives each node its own process — easier to restart individual components and observe per-process CPU/memory.

API Reference

ROS 2 service, topic, and message-type reference for `ros2_kafka_dispatcher`.

Custom message types

introspection_manager_msgs/msg/TopicInfo

```
string name    # ROS 2 topic name, e.g. /camera/image_raw
string type    # ROS 2 message type, e.g. sensor_msgs/msg/Image
```

introspection_manager_msgs/msg/TopicsInfo

```
TopicInfo[] topics    # list of all discovered topics
```

Services

dispatcher_controller

All services are relative to the controller node namespace (default `/dispatcher_controller`).

apply_selection

`dispatcher_controller/srv/ApplySelection`

Applies a GUI-provided topic list. Only valid when `selection_mode` is `gui`; ignored or rejected in other modes.

```
# Request
introspection_manager_msgs/TopicInfo[] topics    # topics to strea
```

```
---
# Response
bool   success
string message
```

reload_selection

`dispatcher_controller/srv/ReloadSelection`

Reload and apply the selection from the current mode: in `file` mode re-reads the YAML file from `selection_file_path`; in `all` mode re-runs topic auto-discovery via `introspection_manager`; in `gui` mode re-applies the last GUI-provided selection if present.

```
 # Request
string selection_file_path   # optional override path (file mode o
bool   apply_now             # immediately apply after reload

---
# Response
bool   success
string message
```

stop_streaming

`dispatcher_controller/srv/StopStreaming`

Deactivate all sinks immediately; no messages will be forwarded until the next `apply_selection` or `reload_selection`.

```
 # Request
bool reset_cached   # also clear cached selections if true

---
# Response
```

```
bool   success
string message
```

get_status

`dispatcher_controller/srv/GetStatus`

Query the current state of the controller and managed sinks.

```
 # Request
(empty)

---
# Response
bool                                    success
string                                  message
string                                  selection_mode
bool                                    streaming_active
string                                  kafka_sink_state
string                                  mosquitto_sink_state
introspection_manager_msgs/TopicInfo[]  applied_topics
uint32                                  gui_selection_count
uint32                                  file_selection_count
uint32                                  all_selection_count
string                                  last_error
builtin_interfaces/Time                 last_error_stamp
bool                                    reconciling
```

set_selection_mode

`dispatcher_controller/srv/SetSelectionMode`

Switch selection mode at runtime.

```
 # Request
string selection_mode        # gui | file | all
string selection_file_path   # path to selection YAML (only for fi
bool   apply_now             # immediately apply after mode change
```

```
---
# Response
bool   success
string message
```

introspection_manager

Relative to the introspection manager node namespace (default `/introspection_manager`).

get_topics

`introspection_manager_msgs/srv/GetTopics`

Return the current snapshot of all discovered topics.

```
 # Request
(empty)

---
# Response
introspection_manager_msgs/TopicInfo[] topics
```

apply_pipeline

`introspection_manager_msgs/srv/ApplyPipeline`

Apply a desired pipeline specification to the dispatcher_controller.

```
 # Request
introspection_manager_msgs/TopicInfo[]      subscriptions       #
introspection_manager_msgs/PluginInstance[] plugins             #
string                                      kafka_mapping_yaml  #

---
# Response
```

```
bool   success
string message
```

get_pipeline_status

`introspection_manager_msgs/srv/GetPipelineStatus`

Return the current pipeline status.

```
 # Request
(empty)

---
# Response
bool                                        success
string                                      message
string                                      kafka_sink_state
introspection_manager_msgs/TopicInfo[]      active_subscriptions
introspection_manager_msgs/PluginInstance[] active_plugins
string                                      last_error
bool                                        reconciling
```

stop_pipeline

`introspection_manager_msgs/srv/StopPipeline`

Stop the current pipeline.

```
 # Request
(empty)

---
# Response
bool   success
string message
```

Published topics

introspection_manager

Topic	Type	QoS
`~/topics_info`	`introspection_manager_msgs/msg/TopicsInfo`	Configurable (default: reliable, volatile)

Published whenever the topic list changes (if `publish_on_change:=true`) or at every poll cycle.

kafka_sink

Topic	Type	Description
`<metrics.topic>`	`std_msgs/msg/String` (JSON)	Per-topic metrics published at `metrics.interval_ms` intervals.

kafka_sink metrics JSON schema

The payload is a JSON **array**. Each element corresponds to one active ROS topic subscription.

```
[
  {
    "ros_topic": "/camera/image_raw",
    "kafka_topic": "ros2.camera.image_raw",
    "msg_type": "sensor_msgs/msg/Image",
    "payload_format": "cdr",
    "interval_ms": 1000,
    "delta": {
      "received": 30,
      "sent_ok": 30,
```

```
    "dropped": 0,
    "errors": 0,
    "bytes": 1382400
  },
  "rates": {
    "received_per_sec": 30.0,
    "sent_per_sec": 30.0
  },
  "message_size": {
    "avg_bytes": 46080.0,
    "min_bytes": 46080,
    "max_bytes": 46080
  },
  "latency_ns": {
    "serialize_avg": 12345,
    "serialize_p95": 15000,
    "serialize_p99": 18000,
    "serialize_max": 20000,
    "send_avg": 5000,
    "send_p95": 7000,
    "send_p99": 9000,
    "send_max": 11000
  },
  "throughput": {
    "serialize_mb_per_sec": 120.5,
    "send_mb_per_sec": 300.0
  },
  "cpu_efficiency": {
    "ns_per_byte": 0.27,
    "bytes_per_cpu_ms": 3700000.0
  },
  "totals": {
    "received": 1234,
    "sent_ok": 1230,
    "dropped": 2,
    "errors": 2,
    "bytes": 56789012
```

```
    }
  }
]
```

kafka_cdr_to_json

Topic	Type	Description
`<metrics.topic>`	`std_msgs/msg/String` (JSON)	Per-topic metrics published at `metrics.interval_ms` intervals.

kafka_cdr_to_json metrics JSON schema

The payload is a JSON **object** with a top-level `topics` array. Each element corresponds to one Kafka input topic being consumed and converted.

```
{
  "timestamp_ns": 1711900000000000000,
  "topics": [
    {
      "input_topic": "ros2.camera.image_raw",
      "output_topic": "ros2.camera.image_raw.json",
      "ros_type": "sensor_msgs/msg/Image",
      "received": 1234,
      "converted": 1230,
      "failed": 4,
      "delta_received": 30,
      "delta_converted": 30,
      "delta_failed": 0,
      "bytes": 56789012,
      "rate_msgs_per_s": 30.0,
      "rate_bytes_per_s": 1382400.0,
      "latency_ns_p50": 2300000,
      "latency_ns_p95": 5100000,
```

```
      "latency_ns_p99": 9800000,
      "latency_ns_max": 15000000,
      "json_size_min": 4096,
      "json_size_max": 49152,
      "json_size_avg": 46000.0
    }
  ]
}
```

Telemetry log format (kafka_sink)

When `telemetry.enabled: true`, the sink logs one JSON line per message (every `telemetry.log_every_n` messages) at `INFO` level via the ROS 2 logger.

```
{
  "msg_id": "...",
  "ros_topic": "/camera/image_raw",
  "kafka_topic": "ros2.camera.image_raw",
  "receive_time_ns": 1711900000000000000,
  "kafka_timestamp_ms": 1711900000000,
  "payload_bytes": 46080,
  "serialize_time_ns": 12345
}
```

Kafka record format

Each Kafka record produced by `kafka_sink` has:

- **Value:** CDR bytes (binary) or JSON string, depending on `kafka.payload_format`.
- **Headers:**

Key	Example value
`ros_type`	`sensor_msgs/msg/Image`

> **Note:** `ros_topic`, `kafka_topic`, `msg_type`, `stamp_ms`, and `payload_format` are **not** Kafka record headers. `ros_topic`, `kafka_topic`, and `payload_format` appear in the telemetry log; `msg_type` appears in the metrics log. The message timestamp is carried as the Kafka record timestamp (set via `timestamp_ms` in `produce()`), not as a header.

Topic naming conventions

Kafka (prefix_ros_topic mode)

ROS topic `/a/b/c` with `kafka.topic_prefix = ros2` becomes Kafka topic `ros2.a.b.c` (slashes replaced with dots, leading slash stripped).

MQTT (prefix_ros_topic mode)

ROS topic `/a/b/c` with `mqtt.topic_prefix = ros2` becomes MQTT topic `ros2/a/b/c` (leading slash stripped, slashes preserved as MQTT hierarchy).

Fixed mode

All messages go to the single configured topic regardless of ROS topic name.

Configuration Reference

Complete parameter reference for all nodes in `ros2_kafka_dispatcher`.

---

dispatcher_controller

Declared in `dispatcher_controller/src/dispatcher_controller_node.cpp`.

Selection

Parameter	Type	Default	Description
`selection_mode`	string	`file`	Topic selection mode: `gui`, `file`, or `all`.
`selection_file_path`	string	`""`	Path to the YAML selection file (required in `file` mode).
`auto_apply_on_mode_change`	bool	`true`	Automatically apply selection when mode changes.
`validate_topics`	bool	`false`	Verify topics exist in the

Parameter	Type	Default	Description
			live graph before activating sinks.

Sink management

Parameter	Type	Default	D
`kafka_sink_node_name`	string	`/kafka_sink`	P q n k li n
`mosquitto_sink_node_name`	string	`/mosquitto_sink`	P q n M s li n
`allow_missing_sinks`	bool	`true`	C o s a
`component_container_name`	string	`/ ros2_kafka_dispatcher_container`	C u t p n

Introspection

Parameter	Type	Default	Description
`introspection_service_name`	string	(derived)	Service name for `~/get_topics`.
`introspection_node_name`	string	(derived)	Node name of the introspection manager.
`disable_introspection_after_apply`	bool	`true`	Stop polling the graph after selection is applied to save CPU.

All-mode

Parameter	Type	Default	Description
`all_mode_max_topics`	int	`200`	Maximum number of topics to select in `all` mode.
`all_mode_allowlist`	string[]	`[]`	Only include topics matching these patterns (empty = allow all).
`all_mode_denylist`	string[]	`[]`	Exclude topics matching these patterns.
`all_mode_hide_rosout`	bool	`true`	

Parameter	Type	Default	Description
			Exclude `/rosout` from auto-selected topics.

Selection file format

```
# dispatcher_controller/config/topics.yaml
- topic_name: /camera/image_raw
  msg_type: sensor_msgs/msg/Image

- topic_name: /cmd_vel
  msg_type: geometry_msgs/msg/Twist
  topic_tools:
    plugin: topic_tools::ThrottleNode
    output_topic: /throttle/cmd_vel
    parameters:
      period: 0.05          # seconds between forwarded messages
```

`msg_type` can be omitted when `validate_topics:=false` and the controller can infer it from the running graph.

---

introspection_manager

Defaults in `introspection_manager/config/introspection_manager.param.yaml`.

Parameter	Type	Default	Description
`publisher_queue_depth`	int	`1`	Queue depth for `~/topics_info` publisher.
`publisher_reliability`	string	`reliable`	QoS reliability: `reliable` or `best_effort`.

Parameter	Type	Default	Description
`publisher_durability`	string	`volatile`	QoS durability: `volatile` or `transient_local`.
`publish_on_change`	bool	`true`	Publish `topics_info` only when the topic list changes.
`filter_hidden`	bool	`true`	Exclude topics whose name segments start with `_`.
`introspection_enabled`	bool	`true`	Enable background graph monitoring thread.

---

kafka_sink

Defaults in `kafka_bridge/kafka_sink/config/kafka_sink.param.yaml`.

Subscriptions / QoS

Parameter	Type	Default	Description
`subscriptions_yaml`	string	`""`	YAML-encoded list of topic subscriptions (set by controller).
`qos_depth`	int	`10`	History queue depth for ROS 2 subscriptions.

Kafka producer

Parameter	Type	Default	Description
`kafka.bootstrap_servers`	string	`localhost:9092`	Comma-separated K broker addresses.
`kafka.client_id`	string	`kafka_sink`	Producer client ident
`kafka.acks`	string	`all`	Producer acknowledgement le (`0` , `1` , or `all`).
`kafka.topic_prefix`	string	`ros2`	Prefix prepended to topic name for Kafka topic (in `prefix_ros_topi` mode).
`kafka.topic_mapping_mode`	string	`prefix_ros_topic`	`prefix_ros_topi` maps `/a/b` → `<prefix>.a.b` ; `fixed` sends everything to `kafka.fixed_top`
`kafka.fixed_topic`	string	`ros2.raw`	Kafka topic used whe `topic_mapping_m` is `fixed` .
`kafka.strict_startup`	bool	`false`	Fail `on_configure` the broker is unreachable.
`kafka.max_queue_messages`	int	`1024`	Maximum messages the in-process send queue.
`kafka.drop_when_full`	bool	`true`	

Parameter	Type	Default	Description
			Drop new messages when queue is full (v blocking).
`kafka.linger_ms`	int	(librdkafka default)	Linger time before sending a batch (ms)
`kafka.batch_size`	int	(librdkafka default)	Maximum bytes per batch.
`kafka.payload_format`	string	`cdr`	Serialization format: `cdr` (binary) or `js`

Metrics

Parameter	Type	Default	Description
`metrics.enabled`	bool	`true`	Publish per-topic metrics to ROS 2.
`metrics.interval_ms`	int	`1000`	Metrics publish interval in milliseconds.
`metrics.topic`	string	`kafka_sink/metrics`	ROS 2 topic name for metrics messages.

Telemetry

Parameter	Type	Default	Description
`telemetry.enabled`	bool	`false`	Log per-message structured

Parameter	Type	Default	Description
			telemetry to the ROS 2 logger.
`telemetry.log_every_n`	int	`1`	Log every Nth message (1 = every message).

Topic subscription YAML format

```
- topic_name: /demo/chatter
  msg_type: std_msgs/msg/String

- topic_name: /camera/image_raw
  msg_type: sensor_msgs/msg/Image
  kafka_name: camera_raw          # optional: override Kafka topic
```

Kafka headers added to every record

Header key	Value
`ros_type`	ROS 2 message type (e.g. `std_msgs/msg/String`)

> **Note:** `ros_topic`, `kafka_topic`, `stamp_ms`, and `payload_format` are included in the telemetry log (logged via `RCLCPP_INFO` when `telemetry_enabled` is true) but are **not** attached as Kafka record headers. The record timestamp is set from the ROS message stamp in milliseconds but is stored as the Kafka record timestamp, not as a header.

---

mosquitto_sink

Defaults in `mosquitto_bridge/mosquitto_sink/config/mosquitto_sink.param.yaml`.

Subscriptions / QoS

Same as `kafka_sink` (`subscriptions_yaml`, `qos_depth`).

MQTT broker

Parameter	Type	Default	Description
`mqtt.broker_host`	string	`localhost`	MQTT broker hostname or IP.
`mqtt.broker_port`	int	`1883`	MQTT broker port.
`mqtt.client_id`	string	`mosquitto_sink`	MQTT client identifier.
`mqtt.username`	string	`""`	MQTT username (leave empty to skip authentication).
`mqtt.password`	string	`""`	MQTT password.
`mqtt.qos`	int	`1`	MQTT QoS level: `0`, `1`, or `2`.
`mqtt.retain`	bool	`false`	Set the MQTT retain flag on published messages.

Parameter	Type	Default	Description
`mqtt.keep_alive_seconds`	int	`60`	MQTT keep-alive interval in seconds.

Topic mapping

Parameter	Type	Default	Description
`mqtt.topic_prefix`	string	`ros2`	Prefix for MQTT topic names (in `prefix_ros_topic` mode).
`mqtt.topic_mapping_mode`	string	`prefix_ros_topic`	Same semantics as `kafka_sink` (`prefix_ros_topic` or `fixed`).
`mqtt.fixed_topic`	string	`ros2/raw`	MQTT topic used when mode is `fixed` .
`mqtt.payload_format`	string	`cdr`	`cdr` or `json` .

TLS / SSL

Parameter	Type	Default	Description
`mqtt.use_tls`	bool	`false`	Enable TLS encryption.
`mqtt.ca_cert_path`	string	`""`	Path to CA certificate file.

Last Will and Testament (LWT)

Parameter	Type	Default	Description
`mqtt.lwt_topic`	string	`""`	LWT topic (empty disables LWT).
`mqtt.lwt_payload`	string	`mosquitto_sink disconnected`	LWT message payload.
`mqtt.lwt_qos`	int	`1`	LWT QoS level.
`mqtt.lwt_retain`	bool	`false`	Set retain flag on LWT message.

Metrics

Same as `kafka_sink` (`metrics.enabled`, `metrics.interval_ms`, `metrics.topic`).

---

kafka_source

Parameter	Type	Default	Description
`kafka.bootstrap_servers`	string	`localhost:9092`	Broker address.
`kafka.group_id`	string	`ros2-kafka-source`	Consumer group ID.
`kafka.topic_pattern`	string	`^ros2\\..*`	Regex pattern for

Parameter	Type	Default	Description
			Kafka topics to consume.
`kafka.offset_reset`	string	`latest`	Offset policy: `earliest` or `latest`.

System-level launch arguments

Exposed by `ros2_kafka_dispatcher_bringup/launch/system_minimal.launch.py`.

Argument	Default	Description
`selection_mode`	`file`	Passed to `dispatcher_cont`
`selection_file_path`	`""`	Path to selection YAM
`kafka_sink_node_name`	`/kafka_sink`	Node name for Kafka
`mosquitto_sink_node_name`	`/mosquitto_sink`	Node name for Mosq sink.
`validate_topics`	`false`	Enable topic validatio
`subscriptions_yaml`	`""`	Direct subscription ov (bypasses controller)
`qos_depth`	`10`	Subscription queue d
`enable_kafka_sink`	`true`	Include Kafka sink in
`enable_mosquitto_sink`	`true`	Include Mosquitto sin launch.

Argument	Default	Description
`controller_log_level`	`debug`	Log level for `dispatcher_cont`
`kafka_sink_log_level`	`info`	Log level for `kafka_`
`mosquitto_sink_log_level`	`info`	Log level for `mosquitto_sink`.
`introspection_manager_log_level`	`DEBUG`	Log level for `introspection_m`
`kafka_sink_param_file`	`""`	Override parameter fi Kafka sink.
`mosquitto_sink_param_file`	`""`	Override parameter fi Mosquitto sink.
`introspection_manager_param_file`	`""`	Override parameter fi introspection manage

Deployment Guide

This document covers Docker-based deployment, broker setup, and production considerations.

Prerequisites

- ROS 2 (Humble or later) installed on the target machine, or use the provided Docker image.
- Apache Kafka broker reachable from the ROS 2 machine.
- (Optional) MQTT broker reachable from the ROS 2 machine.

Building the Docker image

The repository ships a multi-stage `Dockerfile` at `docker/Dockerfile`.

```
docker build \
  -f docker/Dockerfile \
  --build-arg ROS_DISTRO=humble \
  --build-arg GIT_SHA="$(git rev-parse HEAD)" \
  --build-arg BUILD_DATE="$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  --build-arg IMAGE_SOURCE="https://github.com/<org>/ros2_kafka_di
  -t ros2-kafka-dispatcher:local \
  .
```

Smoke test

```
docker run --rm ros2-kafka-dispatcher:local \
  bash -lc "source /opt/ros/humble/setup.bash && \
```

```
            source /ws/install/setup.bash && \
            ros2 pkg list >/dev/null && echo OK"
```

Starting a local Kafka broker (development)

A `docker-compose.yml` for a single-broker Kafka cluster is provided in `kafka_bridge/kafka_brocker/`.

```
cd kafka_bridge/kafka_brocker
docker compose up -d
```

The broker listens on `localhost:9092` by default.

Verify connectivity:

```
# Install kcat (formerly kafkacat) for quick checks
kcat -b localhost:9092 -L
```

Starting a local MQTT broker (development)

A `docker-compose.yml` for Mosquitto is provided in `mosquitto_bridge/mosquitto_brocker/`.

```
cd mosquitto_bridge/mosquitto_brocker
docker compose up -d
```

The broker listens on `localhost:1883` by default.

Minimal system launch

After building and sourcing the workspace:

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  selection_mode:=file \
  selection_file_path:=/path/to/topics.yaml
```

This starts: - `introspection_manager` - `dispatcher_controller` - `kafka_sink` - `mosquitto_sink`

To disable either sink:

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  selection_mode:=file \
  selection_file_path:=/path/to/topics.yaml \
  enable_mosquitto_sink:=false
```

Composed (single container) launch

Runs all nodes as components in one process — lower IPC overhead, easier to monitor as a single PID.

```
ros2 launch ros2_kafka_dispatcher_bringup system_composed.launch.p
  selection_mode:=file \
  selection_file_path:=/path/to/topics.yaml
```

Production parameter files

For production deployments, override default parameters with YAML files instead of command-line arguments.

kafka_sink.param.yaml (example)

```
kafka_sink:
  ros__parameters:
    qos_depth: 50
    kafka:
      bootstrap_servers: "kafka-broker-1:9092,kafka-broker-2:9092"
```

```
      client_id: "robot-1-kafka-sink"
      acks: "all"
      topic_prefix: "robot1"
      topic_mapping_mode: "prefix_ros_topic"
      payload_format: "cdr"
      strict_startup: true
      max_queue_messages: 4096
      linger_ms: 5
      batch_size: 1048576
    metrics:
      enabled: true
      interval_ms: 5000
```

Pass to the launch:

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  kafka_sink_param_file:=/path/to/kafka_sink.param.yaml
```

mosquitto_sink.param.yaml (example with TLS)

```
mosquitto_sink:
  ros__parameters:
    mqtt:
      broker_host: "mqtt.example.com"
      broker_port: 8883
      client_id: "robot-1-mqtt-sink"
      username: "ros2"
      password: "secret"
      qos: 1
      use_tls: true
      ca_cert_path: "/etc/ssl/certs/ca-certificates.crt"
      lwt_topic: "robot1/status"
      lwt_payload: "offline"
      lwt_retain: true
```

Docker Hub / GHCR publishing (CI/CD)

The GitHub Actions workflow in `.github/workflows/` builds the image and publishes it:

- **Pull requests / pushes to `main`:** Build + smoke test only.
- **Tags matching `v*.*.*`:** Publish to GHCR and Docker Hub.

Required GitHub secrets:

Secret	Description
`DOCKERHUB_USERNAME`	Docker Hub username or bot account.
`DOCKERHUB_TOKEN`	Docker Hub access token.
`DOCKERHUB_REPOSITORY`	Full image name, e.g. `your-org/ros2-kafka-dispatcher`.

Publish a release:

```
git tag v1.2.3
git push origin v1.2.3
```

The workflow publishes: - `your-org/ros2-kafka-dispatcher:1.2.3` - `your-org/ros2-kafka-dispatcher:sha-<shortsha>` - `your-org/ros2-kafka-dispatcher:latest` (from `main` only)

Production checklist

- [ ] Set `kafka.strict_startup: true` so the node fails clearly if the broker is unreachable at startup.
- [ ] Configure `kafka.acks: all` and appropriate `kafka.max_queue_messages` for your throughput requirements.

- [ ] Use a dedicated Kafka `client_id` per robot/deployment to distinguish producers in broker metrics.
- [ ] Set `all_mode_max_topics` if using `all` mode to prevent accidentally subscribing to thousands of topics.
- [ ] Enable `metrics.enabled` and pipe metrics to your monitoring system.
- [ ] Use `system_composed.launch.py` for simpler process supervision (single PID per deployment).
- [ ] Synchronise system clocks (NTP/PTP) if you need accurate end-to-end latency measurements.
- [ ] Prefer `payload_format: cdr` for bandwidth efficiency; use `json` only when downstream consumers cannot handle CDR.

Developer Guide

How to build, test, extend, and contribute to `ros2_kafka_dispatcher`.

Repository layout

```
ros2_kafka_dispatcher/
├── dispatcher_controller/          # Control-plane orchestration
├── introspection_manager/          # Graph monitoring node
├── introspection_manager_msgs/     # Custom message and service d
├── kafka_bridge/
│   ├── kafka_client/               # C++ librdkafka wrapper (libr
│   ├── kafka_sink/                 # ROS 2 → Kafka lifecycle node
│   ├── kafka_source/               # Kafka → ROS 2 lifecycle node
│   ├── kafka_cdr_to_json/          # CDR → JSON converter node
│   ├── kafka_brocker/              # Docker Compose for local bro
│   └── kafka_mock_server/          # Python mock broker for testi
├── mosquitto_bridge/
│   ├── mosquitto_sink/             # ROS 2 → MQTT lifecycle node
│   └── mosquitto_brocker/          # Docker Compose for local bro
├── plugin_interfaces/              # Plugin interface definitions
├── plugin_loader/                  # Plugin loader (stub)
├── processing_plugins/             # Example plugin packages (pla
├── ros2_kafka_dispatcher_bringup/  # System-level launch files an
├── tools/
│   └── latency/                    # Latency measurement scripts
├── docker/                         # Multi-stage Dockerfile
└── docs/                           # Documentation (this director
```

Build

```
# From your ROS 2 workspace root
rosdep install --from-paths src --ignore-src -y
colcon build --symlink-install \
             --cmake-args -DCMAKE_BUILD_TYPE=Release \
                          -DCMAKE_EXPORT_COMPILE_COMMANDS=On
source install/setup.bash
```

Incremental rebuild (faster iteration)

```
colcon build --packages-select dispatcher_controller
source install/setup.bash
```

Debug build

```
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Debug
```

Run tests

```
colcon test --packages-select introspection_manager kafka_sink mos
colcon test-result --verbose
```

Key test files:

Package	Test file
`introspection_manager`	`test/test_introspection_manager.cpp`
`kafka_sink`	`test/test_kafka_sink.cpp`
`mosquitto_sink`	`test/test_mosquitto_sink.cpp`
`gui_client`	`test/` (flake8 / pep257)

Package dependencies overview

Package	Key dependencies
`dispatcher_controller`	rclcpp, rclcpp_lifecycle, rclcpp_components, lifecycle_msgs, introspection_manager_msgs, yaml-cpp
`introspection_manager`	rclcpp, rclcpp_components, introspection_manager_msgs, rcpputils
`kafka_sink`	rclcpp_lifecycle, rclcpp_components, kafka_client, yaml-cpp, rosbag2_cpp
`kafka_client`	librdkafka
`kafka_source`	rclcpp_lifecycle, kafka_client, rosbag2_cpp
`mosquitto_sink`	rclcpp_lifecycle, rclcpp_components, libpaho-mqttpp3, yaml-cpp

Adding a new selection mode

1. Add the mode string to the validation in `dispatcher_controller/src/dispatcher_controller_node.cpp` where `selection_mode` is declared.
2. Add a branch in the mode-switching logic (see `set_selection_mode` service handler).
3. Implement a method that produces a `std::vector<TopicInfo>` and call the existing `applySelection(topics)` helper.

4. Add a launch argument in `ros2_kafka_dispatcher_bringup/launch/system_minimal.launch.py` if the mode needs its own parameters.

Adding a new sink

A sink must be a `rclcpp_lifecycle::LifecycleNode` that:

1. Accepts a `subscriptions_yaml` string parameter listing `{topic_name, msg_type}` pairs.
2. On `on_configure`: parses the YAML and sets up the transport layer.
3. On `on_activate`: creates `rclcpp::GenericSubscription` for each topic and starts sending.
4. On `on_deactivate`: destroys subscriptions and flushes/closes the transport.

Register the new node's fully qualified name as a controller parameter (similar to `kafka_sink_node_name`) and add it to the lifecycle orchestration in `dispatcher_controller_node.cpp`.

Processing plugins (topic_tools)

The controller can inject `topic_tools` plugins between a ROS 2 topic and a sink. Each plugin is loaded as a component into the shared container (`component_container_name`).

Supported topic_tools plugins (ROS 2 Humble+)

Plugin	Purpose
`topic_tools::ThrottleNode`	Rate-limit messages.
`topic_tools::DropNode`	Drop every Nth message.

Plugin	Purpose
`topic_tools::DelayNode`	Introduce a fixed delay.

Selection YAML example

```
- topic_name: /camera/image_raw
  msg_type: sensor_msgs/msg/Image
  topic_tools:
    plugin: topic_tools::ThrottleNode
    output_topic: /throttled/image_raw
    parameters:
      period: 0.1    # forward at most 10 Hz
```

The controller wires `/camera/image_raw` → `ThrottleNode` → `/throttled/image_raw`, then subscribes the sink to `/throttled/image_raw`.

CDR ↔ JSON conversion

kafka_cdr_to_json node

Reads CDR-encoded records from Kafka and republishes as JSON to a new topic (default: appends `.json`).

ros2_kafka_json_sidecar.py

A Python script that does the same conversion as an external consumer:

```
python3 ros2_kafka_json_sidecar.py \
  --bootstrap-servers localhost:9092 \
  --topic-pattern '^ros2\..*' \
  --output-suffix .json \
```

```
  --group-id ros2-json-sidecar \
  --log-level INFO
```

The script reads the `ros_type` Kafka record header to determine the ROS 2 type, deserializes with `rclpy.serialization.deserialize_message`, and publishes JSON.

Latency measurement workflow

See `latency_measurement.md` for a full walkthrough. The quick version:

```
# Start the full pipeline
./tools/latency/run_latency_capture.sh \
  --count 1000 \
  --rate 50 \
  --payload-bytes 256 \
  --output-dir ./latency_run_1

# Artifacts
# latency_run_1/publisher.jsonl   - per-message t0 timestamps
# latency_run_1/consumer.jsonl    - per-message t0, t1, latency_ns
# latency_run_1/kafka_sink.log    - per-message telemetry from the
```

Code style

- C++: follow the ROS 2 coding style (clang-format, ament_lint).
- Python: PEP 8 (enforced via flake8 and pep257 tests in `gui_client`).
- Run linters before opening a PR:

```
 ament_flake8 gui_client
ament_pep257 gui_client
ament_cpplint dispatcher_controller
```

Common pitfalls

Symptom	Likely cause
Sink stays `INACTIVE` after apply	Broker unreachable; check `kafka.bootstrap_servers` / `mqtt.broker_host`.
`get_status` shows `ERROR` phase	Call `get_status` and inspect `last_error`.
Topics missing from introspection	`introspection_manager` not running, or `filter_hidden:=true` hiding the topic.
Messages dropping at high rate	`kafka.max_queue_messages` too low or broker too slow; increase queue or enable batching.
Unknown type during serialization	Ensure the message package is on `AMENT_PREFIX_PATH` and the node can load the type library.

Benchmark Plan

This document defines the test matrix, tooling, procedure, and result schema for benchmarking the ROS 2 → Kafka → ROS 2 pipeline.

For metric definitions and measurement rules, see `performance_metrics.md`. For clock synchronization and telemetry setup, see `latency_measurement.md`.

---

1. Test matrix

1.1 Payload sizes

Use a logarithmic spread that covers small control messages through large sensor payloads:

Tier	Size	Typical ROS 2 equivalent
XS	256 B	`std_msgs/String`, `geometry_msgs/Twist`
S	1 KB	`sensor_msgs/Imu`, `nav_msgs/Odometry`
M	10 KB	`sensor_msgs/LaserScan` (mid-range)
L	100 KB	`sensor_msgs/PointCloud2` (sparse)
XL	1 MB	`sensor_msgs/Image` (uncompressed, stress only)

Run XL only to find the saturation ceiling — it is not expected to be sustainable at high rates.

1.2 Message rates

Label	Rate	Purpose
Low	100 msg/s	Baseline, establishes minimum overhead
Mid	1 000 msg/s	Moderate load
High	5 000 msg/s	Near-saturation for most payload sizes
Max	10 000 msg/s	Include only if the pipeline sustains it without drops

Compact grid (if time is limited): **100 / 1k / 10k**.

1.3 Message schemas

Flat (simple primitives)

```
uint64 id
float64 ts
float32[] values   # sized to hit payload target
```

Nested (structured)

```
header { uint64 id, float64 ts }
pose   { float32[3] pos, float32[4] quat }
meta   { uint32 flags, uint32[] tags }
```

Use both **fixed-size** (`float32[64]`) and **variable-size** (`float32[]`) arrays for each payload tier. Adjust array lengths to hit the target byte count.

1.4 Full grid

Each cell = (payload size) × (rate) × (schema) × (array variant). With the compact rate set and both schemas:

	100 msg/s	1k msg/s	10k msg/s
256 B flat fixed	✓	✓	✓
256 B nested var	✓	✓	✓
1 KB flat fixed	✓	✓	✓
…	…	…	…
100 KB nested var	✓	✓	(if feasible)

---

2. Fixed variables (held constant across all runs)

Kafka producer

Parameter	Value
`kafka.acks`	`all`
`kafka.linger_ms`	`5`
`kafka.batch_size`	`1048576` (1 MB)
`kafka.max_queue_messages`	`4096`
`kafka.payload_format`	`cdr`
Partitions	1 (single-partition topic)
Replication factor	1 (local broker)
Compression	none (add as a separate variable if needed)

ROS 2 QoS

Setting	Value
Reliability	`reliable`
Durability	`volatile`
History	`keep_last`
Depth	`50`

Environment

- Same host machine and kernel version for all runs.
- CPU governor set to `performance` : `sudo cpupower frequency-set -g performance`
- No other significant workloads running.
- Kafka and the ROS 2 process on the same machine (eliminates network variable).

---

3. Procedure

3.1 Start brokers

```
cd kafka_bridge/kafka_brocker && docker compose up -d
```

Verify:

```
kcat -b localhost:9092 -L
```

3.2 Configure kafka_sink

Enable telemetry in your `kafka_sink.param.yaml` :

```
kafka_sink:
  ros__parameters:
```

```
    kafka:
      bootstrap_servers: "localhost:9092"
      payload_format: "cdr"
      acks: "all"
      linger_ms: 5
      batch_size: 1048576
      max_queue_messages: 4096
    telemetry:
      enabled: true
      log_every_n: 1
    metrics:
      enabled: true
      interval_ms: 1000
```

3.3 Launch the pipeline

```
ros2 launch ros2_kafka_dispatcher_bringup system_minimal.launch.py
  selection_mode:=file \
  selection_file_path:=/path/to/bench_topics.yaml \
  kafka_sink_param_file:=/path/to/kafka_sink.param.yaml \
  enable_mosquitto_sink:=false
```

3.4 Run a single benchmark cell

Use the latency runner from `tools/latency/`:

```
./tools/latency/run_latency_capture.sh \
  --count 6000 \
  --rate 1000 \
  --payload-bytes 10240 \
  --output-dir ./results/2025-04-01__size-10240__rate-1000__schema
```

Flags:

Flag	Description
`--count`	

Flag	Description
	Total messages to send (use `rate × (warmup + measurement)` seconds)
`--rate`	Target publish rate in msg/s
`--payload-bytes`	Serialized payload target size
`--output-dir`	Directory for artifacts (created automatically)

3.5 Artifacts per run

File	Contents
`publisher.jsonl`	`msg_id`, `t0_ns`, payload size, topic
`consumer.jsonl`	`msg_id`, `t0_ns`, `t1_ns`, `latency_ns`, payload size
`kafka_sink.log`	Per-message telemetry (serialize time, payload bytes)
`kafka_source.log`	Consumer-side log

---

4. Duration and repetitions

Phase	Duration
Warm-up (excluded from analysis)	30 s
Measurement window	3 min (180 s)

- **5 runs minimum** per grid cell.
- Increase to **10 runs** if p99 latency variance exceeds 20% across runs.
- Discard the first run if the broker JVM needs warming up.

---

5. Metrics to record per cell

From `consumer.jsonl` (drop the warm-up period messages before aggregating):

Metric	Aggregation
End-to-end latency	mean, p50, p95, p99 (ms)
Throughput	sustained mean msg/s and MB/s
Drop rate	`(sent - received) / sent`
Payload size	mean bytes (verify against target)

From system monitoring during the run:

Metric	Tool
CPU (per process)	`pidstat -p <pid> 1`
RSS memory	`ps -o pid,rss -p <pid>`
Kafka broker lag	`kcat -b localhost:9092 -G <group> <topic>`

---

6. File naming convention

> `<date>__size-<bytes>__rate-<msgs>__schema-<flat|nested>__array-<fi`

Example:

> `2025-04-01__size-10240__rate-1000__schema-nested__array-var__run-0`

Store all runs for a grid cell in the same parent directory, one subdirectory per run.

---

7. Analysis checklist

- [ ] Strip warm-up messages (first 30 s of `t0_ns` timestamps).
- [ ] Compute latency percentiles per run, then average across runs.
- [ ] Plot latency CDF for each (size, rate) pair.
- [ ] Flag any run with drop rate > 0.1% as a saturated data point.
- [ ] Compare `payload_format: cdr` vs `json` for selected cells if JSON support is needed.
- [ ] Note CPU and memory peaks — these reveal whether the bottleneck is the bridge or the broker.

---

8. Reporting template

For each grid cell, record:

```
 date:             2025-04-01
payload_bytes:    10240
rate_target:      1000
schema:           nested
array_type:       var
runs:             5
warmup_s:         30
measurement_s:    180

latency_mean_ms: 3.2
latency_p50_ms:  2.9
latency_p95_ms:  7.1
latency_p99_ms:  14.3

throughput_msgs_s: 998.4
throughput_mb_s:   9.75

drop_rate_pct:   0.00

cpu_bridge_avg_pct:  12.3
```

```
cpu_bridge_max_pct:  18.7
rss_bridge_mb_avg:   142
rss_bridge_mb_max:   156
```

Performance metrics and measurement definitions

This document defines the primary and secondary performance metrics for the ROS 2 ↔ Kafka dispatcher, along with precise measurement rules, aggregation, and experimental factors to control.

Primary metrics

1. **End-to-end latency (publish → Kafka → consumer receipt)**
2. **Definition:** Time from ROS 2 publisher timestamp to consumer receipt timestamp for the same message.
3. **Data points:** For each message, record `t_ros_publish`, `t_kafka_record`, `t_consumer_recv`.
4. **Latency fields:**
    - `latency_end_to_end = t_consumer_recv - t_ros_publish`
    - `latency_kafka_path = t_consumer_recv - t_kafka_record`
5. **Throughput**
6. **Definition:** Message rate and/or payload rate through the system.
7. **Measurement:**
    - `throughput_msgs = total_messages / test_duration_seconds`
    - `throughput_bytes = total_payload_bytes / test_duration_seconds`
8. **Payload size**

9. **Definition:** Size in bytes of the serialized payload per message.
10. **Measurement:**
    - `payload_bytes_raw` : bytes produced by ROS 2 serialization before Kafka send.
    - `payload_bytes_kafka_record` : size of the Kafka record payload as sent (exclude Kafka protocol overhead).

Secondary metrics

1. **CPU utilization**
2. **Definition:** CPU usage of the ROS 2 publisher, Kafka bridge/sink, and consumer processes.
3. **Measurement:** Average and max CPU across the test window, sampled at a fixed interval (e.g., 1s).
4. **Memory usage (RSS/heap)**
5. **Definition:** RSS and heap usage for the bridge/sink processes.
6. **Measurement:** Average and max memory across the test window, sampled at a fixed interval.
7. **Error rate / drops**
8. **Definition:** Count and ratio of failed operations to total operations.
9. **Measurement:**
    - `serialization_failures`
    - `dropped_messages`
    - `kafka_delivery_errors`
    - `error_rate = (sum of failures) / total_messages`

Precise measurement definitions

Timestamps

- **ROS 2 publish time (`t_ros_publish`)**: Capture from the message header timestamp at publish time. If header timestamps

are absent, log a monotonic timestamp at the publisher immediately before publish.
- **Kafka record timestamp (`t_kafka_record`)**: Capture Kafka `CreateTime` if producer sets record timestamp; otherwise use producer send time captured just before send.
- **Consumer receipt time (`t_consumer_recv`)**: Record a monotonic timestamp immediately upon message reception by the consumer.

Size

- **Raw payload bytes (`payload_bytes_raw`)**: Size of the serialized ROS 2 message before Kafka send.
- **Kafka record payload bytes (`payload_bytes_kafka_record`)**: Size of the record value produced to Kafka. Note whether compression is enabled; if enabled, record compressed bytes and compression type.

Units and aggregation

- **Latency**: milliseconds (ms). Report mean, p50, p95, p99 per test run.
- **Throughput**: messages/sec and bytes/sec. Report sustained average and peak over the test window.
- **Resource usage**: CPU in percent per process; memory in MB. Report average and max over the test window.
- **Error rate**: fraction or percent of total messages.

Experimental factors to record (control variables)

- **Message rate**: target publish rate (msgs/sec) and actual achieved rate.
- **Payload size**: serialized message size (bytes), including distribution if variable.
- **Topics and partitions**: topic count, partition count, replication factor.

- **ROS 2 QoS**: reliability, durability, history depth, deadline/liveliness settings.
- **Kafka producer config**: acks, linger.ms, batch.size, compression.type, retries, buffer.memory.
- **Kafka consumer config**: fetch.min.bytes, fetch.max.bytes, max.poll.records, auto.offset.reset.
- **System setup**: CPU model, core count, memory, OS version, network topology, containerization settings.
- **Test duration and warm-up**: total runtime, warm-up period, and steady-state measurement window.

Latency measurement and telemetry workflow

This guide documents the end-to-end data path, telemetry hooks, and a runnable script to collect structured latency logs for the ROS 2 → Kafka → ROS 2 pipeline.

1) Data path for measurement

Publisher → kafka_sink → Kafka → kafka_source → Consumer

1. **Publisher** injects a unique `msg_id` and a **t0** timestamp into the payload.
2. **kafka_sink** serializes the ROS 2 message and forwards it to Kafka.
3. When telemetry is enabled, it logs per-message serialization time and payload size.
4. **kafka_source** reads the Kafka payload, deserializes it, and republishes on a ROS 2 topic.
5. **Consumer** records **t1** on receive and computes `latency_ns = t1_ns - t0_ns`.

2) Logging hooks / telemetry

kafka_sink telemetry

Enable the structured telemetry logs (per-message) by setting:

```
telemetry.enabled: true
telemetry.log_every_n: 1
```

Each log line is JSON and includes:

- `msg_id`

- `receive_time_ns` (time at the sink callback)
- `kafka_timestamp_ms`
- `payload_bytes`
- `serialize_time_ns`
- `ros_topic` / `kafka_topic`

Producer / consumer logs

The helper scripts below log JSON lines:

- `publisher.jsonl`: `msg_id`, `t0_ns`, payload size, topic
- `consumer.jsonl`: `msg_id`, `t0_ns`, `t1_ns`, `latency_ns`, payload size

3) System monitoring tools

Recommended commands during a run (adjust PIDs from the runner output):

- **CPU**
- `pidstat -p <pid> 1`
- `top -p <pid>`
- `htop` (interactive)
- `perf stat -p <pid>`
- **RSS / memory**
- `ps -o pid,rss,command -p <pid>`
- `cat /proc/<pid>/status | rg -n "VmRSS|VmHWM"`

4) One-shot runner script

Use the provided runner to start:

- Kafka sink (ROS 2 → Kafka)
- Kafka source (Kafka → ROS 2)
- Latency publisher
- Latency consumer

```
./tools/latency/run_latency_capture.sh \
 --count 200 \
```

```
  --rate 10 \
  --payload-bytes 256 \
  --output-dir ./latency_artifacts
```

Artifacts are written as structured JSON lines:

- `publisher.jsonl`
- `consumer.jsonl`
- `kafka_sink.log`
- `kafka_source.log`

5) Clock synchronization

For accurate end-to-end latency:

- **Preferred:** run publisher, sink, source, and consumer on the same machine.
- **Distributed:** synchronize clocks using NTP or PTP. Example NTP check:
- `timedatectl status` (systemd)
- `chronyc tracking` (chrony)